

A lightweight Browser-Based Projection Layer for Runtime-Driven UI Systems

1. Introduction

Modern software increasingly depends on graphical user interfaces that are expected to operate consistently across a wide range of platforms, devices, and execution environments. Despite decades of development in desktop application frameworks, the creation of lightweight, portable, and maintainable user interfaces remains a significant architectural challenge. Existing solutions frequently introduce substantial runtime dependencies, platform-specific abstractions, tightly coupled rendering systems, or deployment overhead which negatively impacts accessibility, maintainability, and long-term extensibility. While more modern alternatives attempt to provide cross-platform functionality, these frameworks often introduce considerable architectural complexity, large runtime footprints, and deeply integrated rendering pipelines. Similar problems can be observed throughout many contemporary UI technologies where application logic, rendering, layout management, lifecycle ownership, and frontend runtime behavior become tightly intertwined.

The previously introduced Data Driven UI Framework proposed a lightweight, ECS-based UI architecture centered around stateless systems, reactive event streams, and strict separation between data, logic, and rendering. Instead of relying on

traditional widget hierarchies or runtime-bound object models, the framework represents UI state as composable component data processed through deterministic layout, input, and rendering passes. Rendering backends remain fully interchangeable by consuming streams of primitive draw commands rather than framework-owned visual objects, enabling platform-independent UI composition with minimal runtime overhead. This paper introduces a complementary browser-based projection and display orchestration layer intended to operate on top of the previously defined runtime architecture. The goal of this additional layer is not to create another browser framework, embedded web runtime, or web application platform. Instead, the proposed system repurposes already installed system browsers as lightweight rendering and interaction runtimes capable of acting as distributed display surfaces.

Modern browsers already provide highly optimized rendering pipelines, GPU acceleration, advanced text layout systems, input handling, networking stacks, accessibility integration, session management, and cross-platform compatibility. These capabilities are available by default on virtually all modern operating systems and mobile devices. However, most browser-based application architectures place significant portions of application ownership inside the browser itself, often leading to duplicated state management, hydration complexity, fragmented execution lifecycles, frontend/backend separation issues, and heavy framework dependencies. The architecture proposed within this paper instead treats the browser as a remotely attachable presentation and interaction node while maintaining authoritative application state, orchestration, synchronization, and execution ownership

entirely within the host runtime. User interfaces are projected into browser instances through synchronized runtime surfaces and reactive stream communication rather than being executed as self-contained browser applications.

The browser is not considered the application runtime itself but instead functions as a reusable projection target capable of rendering runtime-controlled UI surfaces locally or remotely. By leveraging already installed browser engines such as Chromium- and Gecko-based implementations, applications can avoid embedded browser distributions, large deployment sizes, and tightly coupled frontend frameworks while still benefiting from modern rendering capabilities

2. Architectural Motivation

The architecture originates from a practical attempt to reduce the complexity associated with modern graphical application development while preserving portability, modularity, and lightweight deployment characteristics. Rather than introducing yet another standalone browser framework or frontend runtime, the proposed system emerged as an extension of the previously established runtime-centric UI architecture in response to increasingly common requirements for remote tooling, browser-accessible interfaces, mobile interaction surfaces, and distributed runtime visualization, while modern systems already ship with highly capable browser engines which provide many of the low-level capabilities traditionally required by graphical applications.

At the same time, many existing UI solutions continue to rely either on heavyweight native frameworks or on embedded browser runtimes that

substantially increase application complexity and deployment overhead

2.1 Limitations of Existing Desktop UI Frameworks

Despite significant advances in graphical application development, modern desktop UI frameworks continue to suffer from a variety of architectural and operational limitations. Many existing solutions remain tightly coupled to specific operating systems, rendering pipelines, runtime environments, or frontend execution models. As a result, even comparatively small graphical applications frequently inherit substantial complexity through their underlying framework dependencies.

More recent cross-platform frameworks attempt to modernize desktop UI development by introducing platform-independent rendering systems and declarative composition models. However, these frameworks frequently require extensive runtime infrastructure, large dependency graphs, complex build pipelines, and deeply integrated rendering abstractions. While technologically capable, they often introduce operational overhead that can become disproportionate for smaller tools, runtime utilities, dashboards, editors, or lightweight graphical applications. Similar architectural patterns can also be observed throughout many browser-based UI frameworks. Modern frontend ecosystems frequently distribute application ownership across browser runtimes, frontend frameworks, hydration layers, backend APIs, asynchronous state synchronization systems, and virtual rendering abstractions. This fragmentation often results in duplicated state management, increased lifecycle complexity, larger deployment artifacts, and reduced determinism.

Many of these limitations originate from tightly coupled ownership models in which rendering, state management, layout systems, lifecycle control, and interaction handling become inseparable parts of the same runtime abstraction. Therefore we earlier proposed a lightweight ECS-based architecture centered around deterministic state synchronization, renderer abstraction, and reactive runtime-controlled UI composition. The system proposed within this paper intentionally builds upon these concepts rather than attempting to replace them. The browser projection layer introduced here should therefore be understood as a complementary frontend and orchestration solution for the previously defined runtime-centric UI architecture

2.2 Runtime Complexity and Deployment Overhead

The operational complexity associated with many modern UI frameworks frequently extends beyond the framework architecture itself and directly affects application deployment, maintenance, and distribution. Applications relying on embedded browser runtimes, large frontend frameworks, or platform-specific rendering systems often require substantial runtime packages to be distributed alongside comparatively small application binaries.

Solutions such as Electron exemplify this issue by bundling complete browser runtimes together with application-specific logic. While such approaches provide highly capable cross-platform environments, they also introduce significant storage overhead, memory consumption, update complexity, and deployment fragmentation. Even smaller embedded webview-based systems typically depend on platform-specific runtime installation requirements or tightly

coupled rendering backends. This can complicate lightweight tooling scenarios where applications are expected to remain portable, self-contained, rapidly deployable, or directly executable from source-driven build environments.

The architectural goals established by the previously discussed data-driven runtime system emphasize lightweight compilation pipelines, modular runtime composition, minimal external dependencies, and deterministic execution. Introducing large embedded browser runtimes would directly conflict with these goals. This creates a practical architectural problem: modern applications increasingly benefit from browser-grade rendering capabilities while simultaneously attempting to avoid the operational overhead associated with embedded browser distributions and heavyweight frontend application stacks

2.3 Browser Runtimes as Existing System Infrastructure

Modern operating systems already ship with highly capable browser runtimes that provide a large portion of the functionality commonly required by graphical applications. Browser engines such as Chromium and Gecko include GPU-accelerated rendering pipelines, advanced text layout systems, font rendering, networking stacks, accessibility integration, session management, windowing behavior, hardware input handling, and increasingly sophisticated security models.

From a purely technical perspective, many modern browsers already function as highly optimized cross-platform rendering environments. However, contemporary browser application architectures typically treat the browser as the primary application

runtime itself rather than as a reusable rendering subsystem. The architecture proposed within this paper instead reinterprets the browser as reusable system infrastructure. Rather than embedding browser runtimes directly into applications, the proposed system utilizes already installed browsers as externally attachable display and interaction nodes.

This distinction fundamentally changes the relationship between the application runtime and the browser runtime. In conventional web applications, the browser frequently owns:

- application lifecycle management
- substantial portions of application state
- rendering orchestration
- routing logic
- execution scheduling
- and frontend interaction behavior

Within the proposed architecture, these responsibilities remain entirely under host runtime ownership. The browser instead acts as a synchronized projection target capable of rendering runtime-controlled UI surfaces. This model intentionally avoids treating browser runtimes as authoritative application containers. Instead, browsers are repurposed as lightweight projection environments capable of presenting synchronized runtime surfaces through standardized communication mechanisms.

One of the most important technologies enabling this approach is the WebSocket protocol. Unlike traditional stateless HTTP request-response communication, WebSockets provide persistent bidirectional

communication channels between browser clients and host runtimes. This allows runtime-controlled UI graphs, interaction events, synchronization messages, and display updates to be exchanged continuously with minimal overhead. Through persistent synchronization channels, browser instances become reactive display nodes rather than autonomous frontend runtimes

2.4 Separation of Runtime and Presentation

The architectural separation between runtime ownership and presentation ownership represents one of the central concepts of the proposed system. Traditional browser applications frequently place substantial portions of execution behavior inside the browser itself. Frontend frameworks commonly manage application state, interaction routing, lifecycle scheduling, layout synchronization, asynchronous communication, and rendering orchestration directly within browser-controlled execution environments.

This architecture introduces several long-term problems:

- fragmented ownership models
- duplicated application state
- increased synchronization complexity
- frontend/backend divergence
- reduced determinism
- and complicated reconnect or session persistence behavior

The proposed system instead treats the browser exclusively as a projection and interaction surface. Authoritative application state remains entirely controlled by the host runtime. Browser clients receive synchronized projections of runtime-controlled UI graphs and emit interaction events back to the runtime through reactive communication channels.

As a result, browser clients do not function as self-contained applications but rather as synchronized runtime surfaces attached to a centralized orchestration layer. This architecture closely aligns with the renderer abstraction principles established within the earlier Data Driven UI Framework. Rendering backends remain interchangeable because the browser projection layer consumes synchronized runtime state rather than framework-owned visual objects or embedded frontend logic. The browser therefore becomes only one possible rendering backend among multiple potential projection targets

2.5 Motivation for Distributed Display Surfaces

The increasing demand for lightweight remote tooling and multi-device interaction environments further motivates the separation between runtime orchestration and display projection. Many modern workflows benefit from distributed interaction surfaces that extend beyond traditional desktop windowing models. Examples include remote dashboards, mobile control panels, browser-accessible runtime tooling, secondary monitoring surfaces, collaborative runtime interfaces, and tablet-based interaction systems.

Particularly within development and production environments, lightweight remote surfaces can provide significant

usability advantages without requiring full remote desktop systems or dedicated native applications. A tablet device operating as a browser-based control surface may, for example, provide large interaction elements, readable monitoring information, or runtime management interfaces while remaining synchronized with a centralized host runtime.

Because the proposed architecture maintains authoritative runtime ownership entirely outside the browser, browser clients may connect, disconnect, reconnect, or move across devices without fundamentally disrupting application execution. Runtime state persists independently of individual display surfaces. This model naturally extends toward distributed runtime interaction scenarios in which multiple browser clients simultaneously observe or interact with synchronized runtime surfaces

2.6 Transition Toward a Display Server Architecture

As browser instances increasingly function as externally attachable runtime surfaces rather than self-contained applications, the overall system architecture begins to resemble a modernized display server model. Traditional display servers such as X11 historically focused on window composition, graphical output management, and remote display capabilities. The proposed architecture extends these ideas toward runtime-centric orchestration of synchronized interaction surfaces.

The proposed display server abstraction is therefore not limited to window management alone. Instead, it acts as a runtime-controlled orchestration layer responsible for:

- synchronized UI surface projection

- browser client management
- capability negotiation
- interaction routing
- session persistence
- reactive stream synchronization
- and distributed display coordination

Browser engines such as Chromium and Gecko become interchangeable projection runtimes attached to this orchestration layer rather than directly embedded application frameworks. This architectural model allows lightweight graphical applications, remote tooling environments, runtime dashboards, and browser-accessible interaction systems to operate without shipping dedicated browser runtimes or depending on heavyweight frontend application stacks.

The following chapters introduce the display server abstraction in greater detail and examine the runtime synchronization, browser integration, and projection mechanisms required to support distributed browser-based runtime surfaces across desktop and mobile environments

3. Display Server Abstraction

The separation between runtime orchestration and graphical presentation forms the foundation of the proposed display architecture. Rather than tightly coupling rendering behavior, frontend execution, synchronization logic, and application ownership into a single framework runtime, the system instead introduces a generalized display server abstraction responsible exclusively for

projection, synchronization, interaction routing, and display surface management.

The display server does not attempt to function as a complete UI framework, rendering engine, or application runtime. Instead, it acts as an independent orchestration layer capable of managing synchronized runtime surfaces independently of the specific systems responsible for generating them. A particularly important consequence of this approach is that the display server remains intentionally decoupled from any framework. Instead, all incoming runtime representations are translated into a generalized surface model understood by the display orchestration layer.

This separation allows the display infrastructure to operate independently from the completion state of the surrounding UI framework while simultaneously preserving long-term compatibility with the larger runtime architecture. Lightweight tools, dashboards, control surfaces, runtime utilities, and remote interaction systems may therefore already operate on top of the display server abstraction without requiring a complete UI framework implementation to exist

3.1 Generalized Display Server Model

Traditional display servers historically focused on low-level graphical composition, window management, display synchronization, and remote rendering. While the proposed system inherits several conceptual similarities from these architectures, its primary responsibility is not direct rendering ownership but runtime surface orchestration. The display server therefore acts as an intermediary layer between runtime-generated surface

descriptions and externally attachable projection environments such as browser-based display clients.

At the highest level, the architecture may be represented conceptually as:

Runtime Sources ->
Translation Layer ->
Display Server ->
Projection Runtime ->
Display Nodes

Within this model, the display server assumes responsibility for:

- synchronized surface management
- runtime surface registration
- session ownership
- state synchronization
- interaction routing
- capability negotiation
- projection lifecycle management
- and distributed display coordination

Importantly, the display server does not directly own application state or application execution. Instead, the authoritative state remains under runtime ownership while the display server manages synchronized projections of externally controlled runtime surfaces. This distinction allows the display server to remain lightweight, modular, and independent from the application-specific runtime architecture generating the surfaces

3.2 Runtime Surface Abstraction

In order to remain independent from specific frontend frameworks or ECS ownership models, the display server operates on a generalized runtime surface abstraction.

A surface represents a synchronized projection target capable of exposing runtime-controlled visual state and receiving interaction events. Surfaces may represent complete application windows, remote dashboards, embedded control panels, runtime overlays, mobile interaction layouts, or arbitrarily nested presentation regions.

The display server itself does not require knowledge regarding the internal runtime structures responsible for generating these surfaces. Instead, surfaces are represented through generalized state descriptions consisting of:

- hierarchical element structures
- layout information
- styling data
- interaction bindings
- synchronization metadata
- and capability descriptors

This abstraction intentionally avoids direct dependence on framework-owned widget hierarchies or renderer-specific object models. A surface therefore behaves as a synchronized state projection rather than a self-contained application instance.

Because surfaces remain detached from specific rendering implementations, the same synchronized surface representation may be projected into:

- browser runtimes
- native rendering backends
- remote display clients
- mobile devices
- embedded overlays
- or alternative presentation environments

This abstraction forms the foundation required for runtime-independent projection behavior

3.3 Translation Layer Architecture

The display server intentionally separates runtime-specific UI generation from generalized display orchestration through a dedicated translation layer. This translation layer acts as an adapter between arbitrary runtime architectures and the generalized surface model consumed by the display server.

Within ECS-driven environments, entities, components, layout systems, interaction systems, and rendering systems may collectively generate synchronized surface state which is then translated into the generalized surface representation understood by the display server.

However, the same display infrastructure may also receive surface descriptions originating from:

- standalone runtime utilities
- manually constructed surface definitions
- scripting environments

- JSON-based surface descriptions
- remote runtime systems
- reactive stream pipelines
- or custom application-specific generators

The display server itself remains completely unaware of the originating runtime model. This architectural separation preserves several important properties:

- framework independence
- modular extensibility
- renderer abstraction
- lightweight tooling support
- and incremental runtime adoption

Most importantly, this model allows the display infrastructure to remain immediately usable even while surrounding runtime systems continue to evolve independently. The translation layer therefore acts as a compatibility boundary rather than a framework dependency

3.4 Runtime-Owned Surface Graphs

Internally, the display server manages synchronized runtime surfaces through retained surface graphs. A surface graph represents the hierarchical relationship between synchronized surfaces, visual elements, interaction regions, session ownership information, and synchronization metadata.

Unlike traditional widget hierarchies, the surface graph does not contain

framework-owned rendering logic or autonomous frontend behavior. Instead, it acts purely as a synchronized projection structure representing an externally owned runtime state.

Each node within the graph maintains stable identity information allowing deterministic synchronization, incremental patch generation, interaction routing, and reconnect-safe state restoration. This retained graph structure enables the display server to:

- synchronize only incremental state changes
- preserve interaction continuity across reconnects
- support distributed display projections
- maintain deterministic synchronization order
- and coordinate multiple projection runtimes simultaneously

Because the graph itself remains renderer-independent, the display server can project identical runtime surfaces into multiple browser clients or alternative rendering environments without duplicating runtime ownership. The retained surface graph therefore functions as the synchronization backbone of the projection system

3.5 Projection Surfaces and Display Nodes

Projection runtimes act as externally attachable environments responsible for rendering synchronized runtime surfaces. Within the proposed architecture, browser

instances function as display nodes connected to the display server through persistent synchronization channels. A display node therefore represents:

- a projection runtime
- an interaction endpoint
- and a synchronized rendering surface

Display nodes may operate locally or remotely and may attach or detach dynamically during runtime execution. Because authoritative state ownership remains outside the browser runtime, display nodes may reconnect or migrate between devices without fundamentally affecting runtime execution.

This architecture differs significantly from conventional browser application models where browser instances frequently own substantial portions of application state and execution lifecycle management. Within the proposed system, browser runtimes instead act as lightweight reactive projection environments synchronized through externally controlled runtime states.

Multiple display nodes may simultaneously observe or interact with synchronized runtime surfaces depending on capability permissions and session ownership policies.

This model naturally enables:

- remote dashboards
- mobile control surfaces
- secondary interaction displays
- collaborative runtime visualization

- and distributed tooling environments

3.6 Reactive Synchronization Model

Synchronization between runtime sources, the display server, and projection runtimes occurs through persistent reactive communication channels. Rather than repeatedly transferring complete frontend state snapshots, the system synchronizes incremental runtime changes through continuous stream-oriented communication. Persistent bidirectional communication channels such as WebSockets allow display nodes to:

- receive synchronized surface updates
- emit interaction events
- negotiate capabilities
- report display state changes
- and maintain session continuity

The synchronization layer therefore behaves similarly to a reactive event stream connecting runtime-owned state with externally projected display surfaces. Incremental synchronization significantly reduces bandwidth usage while simultaneously improving responsiveness and reconnect behavior.

Because display nodes do not own authoritative runtime state, synchronization remains deterministic and reconnect-safe. Browser clients may disconnect and later reconstruct synchronized surface state directly from the runtime-owned surface graph. This architecture avoids many of the lifecycle and synchronization problems

commonly associated with traditional browser frontend applications

3.7 Input Routing and Interaction Flow

Interaction events originating from display nodes are routed back into the runtime environment through the display server orchestration layer. Display nodes therefore function primarily as:

- rendering environments
- interaction capture surfaces
- and synchronization endpoints

Input ownership remains externally controlled. Pointer events, keyboard interaction, touch gestures, focus changes, resize events, and capability state changes are transmitted back to the runtime through persistent synchronization channels where they are processed by runtime-owned interaction systems.

This architecture preserves deterministic interaction handling while avoiding fragmented frontend/backend ownership models. The display server acts as the coordination boundary responsible for:

- session-aware event routing
- interaction ownership validation
- capability-aware input handling
- and synchronized interaction propagation

Because interaction behavior remains runtime-owned, browser runtimes themselves remain lightweight and largely

stateless beyond temporary presentation state

3.8 Browser Runtime Independence

The proposed architecture intentionally avoids direct dependence on any single browser engine or browser-specific frontend framework. Browser runtimes are treated as interchangeable projection environments connected through standardized communication and synchronization mechanisms.

This abstraction layer allows projection runtimes based on Chromium, Gecko, or future browser engines to operate within the same generalized synchronization model. Particular emphasis is placed on avoiding assumptions that would tightly couple the system to Chromium-specific APIs or frontend execution behavior.

Instead, the architecture prioritizes:

- standards-based browser communication
- browser capability negotiation
- renderer-independent synchronization
- and lightweight projection behavior

This design allows the projection system to remain portable across desktop and mobile environments while preserving long-term browser engine flexibility. The following chapters further examine the synchronization behavior, browser integration layers, and runtime projection mechanisms required to implement lightweight distributed display surfaces across browser-based display environments

4. Reactive Surface Synchronization

The display server architecture relies on continuous synchronization between runtime-owned surface graphs and externally attached projection environments. Rather than transmitting complete frontend state snapshots or executing autonomous application logic inside browser runtimes, the architecture preserves authoritative runtime ownership while simultaneously allowing browser-based display nodes to maintain lightweight synchronized projections of runtime-controlled surfaces. The resulting model behaves more similarly to a distributed composition system than to a conventional browser application architecture.

A central design goal of the synchronization layer is the minimization of frontend complexity. Projection runtimes should remain lightweight, deterministic, reconnect-safe, and largely independent from application-specific execution behavior. As a result, synchronization is intentionally based around a small number of generalized structural operations operating on retained synchronized element graphs

4.1 Persistent Synchronization Channels

Synchronization between runtime systems, display servers, and browser-based projection runtimes occurs through persistent bidirectional communication channels.

Unlike traditional stateless HTTP request-response communication models, persistent synchronization channels allow

runtime state and interaction events to flow continuously between connected systems. The architecture primarily relies on WebSockets as the transport mechanism for runtime synchronization.

WebSockets provide low-overhead full-duplex communication channels capable of transmitting incremental surface updates, interaction events, capability negotiation messages, and session synchronization data without repeatedly establishing new network connections.

Within the proposed architecture, browser runtimes do not behave as autonomous frontend applications but instead operate as synchronized projection environments attached to runtime-owned surface graphs. Persistent synchronization channels therefore serve several important purposes:

- incremental structural synchronization
- runtime interaction routing
- capability negotiation
- reconnect-safe session restoration
- projection lifecycle coordination
- and distributed surface synchronization

Because the browser runtime itself does not own authoritative application state, synchronization remains deterministic and resilient against temporary connection interruptions

4.2 Retained Surface Graph Synchronization

Internally, synchronized UI structures are represented through retained hierarchical surface graphs. Each synchronized node within the graph represents either:

- a generic UI element
- a composable surface region
- or a specialized projection structure

Surfaces themselves are therefore treated as specialized synchronized elements capable of hosting nested child elements and additional projection semantics. A display node always exposes a root surface representing the primary projection space of the connected runtime environment. Additional child surfaces may exist beneath the root surface in order to represent:

- application windows
- overlays
- taskbars
- popup menus
- floating panels
- modal regions
- or embedded runtime projections

This model allows the synchronization system to represent both lightweight application layouts and more advanced workspace-oriented composition systems through the same generalized hierarchy. Because synchronized surfaces behave as retained runtime-owned structures, browser runtimes may disconnect and later

reconstruct synchronized state directly from the authoritative runtime graph

4.3 Handle-Based Element Identity

All synchronized structures within the projection system are identified through stable runtime-generated handles. Rather than relying on string-based identifiers or framework-owned object references, the synchronization model uses lightweight numeric handles representing:

- synchronized element instances
- composable surfaces
- interaction regions
- and projection structures

This approach significantly reduces synchronization overhead while simultaneously improving deterministic state reconstruction and runtime lookup performance.

The root surface of a display node is represented through a reserved null-equivalent handle comparable to root composition spaces used by traditional windowing systems.

Child surfaces and synchronized elements maintain explicit parent-child relationships through retained handle references.

This structure allows:

- reconnect-safe synchronization
- incremental structural patching
- deterministic element lookup

- lightweight projection state reconstruction
- and efficient runtime-side graph traversal

Because all synchronized structures share the same generalized handle model, normal UI elements and composable surfaces may be synchronized through identical patch operations

4.4 Incremental Structural Operations

Synchronization between runtime systems and projection runtimes is performed through batched structural update messages. Instead of transmitting individual object-based messages or standalone JSON objects, the protocol is designed around array-encoded command packets. This format reduces serialization overhead, improves parsing efficiency, and aligns naturally with batch-oriented runtime updates.

Each synchronization message follows the structure:

```
["command", { args }]
```

Multiple operations may be transmitted as sequential entries within a single batch payload, allowing efficient aggregation of structural updates within a single transmission cycle. The architecture intentionally minimizes protocol complexity by restricting all interactions to a small set of generalized structural operations operating on retained element graphs

Append Operation

The append operation creates a new synchronized element instance from a registered template and attaches it to a parent element within the retained graph.

Parameters:

- **id** Registered template identifier
- **handle** Instance handle of the newly created element
- **parent** Optional parent handle. A value of 0 represents the root surface

Example:

```
["append", { "id": 10, "handle": 4096, "parent": 0 }]
```

Move Operation

The move operation reattaches an existing element to a new parent within the retained graph.

Parameters:

- **handle** Instance handle of the element being moved
- **parent** Optional new parent handle. A value of 0 represents the root surface

Example:

```
["move", { "handle": 4096, "parent": 2048 }]
```

Modify Operation

The modify operation updates properties of an existing element instance.

Parameters:

- **handle** Instance handle of the element being modified
- **properties** Key-value map of property updates

Example:

```
["modify", { "handle": 4096, "properties": { "text": "Connected", "visible": true } }]
```

Remove Operation

The remove operation deletes an existing element instance from the retained graph.

Parameters:

- **handle** Instance handle of the element being removed

Example:

```
["remove", { "handle": 4096 }]
```

These operations form a minimal, deterministic synchronization core capable of expressing hierarchical UI state changes while remaining independent from rendering backends or frontend frameworks

4.5 Template-Based Element Instantiation

Projection runtimes instantiate synchronized elements using pre-registered templates. HTML5 template elements provide a browser-native mechanism for storing pre-parsed DOM structures that can be cloned efficiently at runtime without repeated construction overhead.

Instead of transmitting raw HTML or DOM instructions, append operations reference template identifiers that are resolved locally within the projection runtime. This approach reduces allocation overhead, minimizes layout recalculations, and enables efficient batch instantiation of synchronized UI structures

4.6 Deterministic Runtime Ownership

A strict separation exists between runtime-owned state and projection-owned state. Projection runtimes do not maintain authoritative application logic or persistent state. Instead, they render synchronized views derived from runtime-owned surface graphs. This guarantees deterministic behavior across reconnects and eliminates frontend/backend state divergence

4.7 Session Persistence and Reconnection

Because synchronization is based on retained state graphs and incremental command streams, projection runtimes can reconnect and reconstruct UI state without full application restart. Upon reconnection, the runtime replays or rehydrates the current surface graph state using the same command model.

4.8 Capability Negotiation and Projection Coordination

Display nodes negotiate capabilities during session initialization. Capabilities influence rendering behavior, interaction handling, and feature activation. Examples include display constraints, input methods, and browser feature availability.

This ensures portability across different browser engines and device classes while maintaining consistent synchronization semantics

5. Browser Projection Runtime

The browser projection runtime acts as the client-side execution environment responsible for rendering synchronized runtime surfaces, forwarding interaction events, and maintaining lightweight local projection state. Unlike conventional browser applications, the projection runtime does not behave as an autonomous frontend framework. Instead, it operates as a deterministic projection layer attached to a runtime-owned synchronization system.

A central architectural goal of the projection runtime is the strict separation between static runtime infrastructure and dynamic synchronized state. Templates, styling rules, event bindings, and local runtime functionality are generated during display server startup and delivered as a stable projection environment to connected browser clients. Runtime synchronization is then limited exclusively to structural state updates and interaction events. This separation significantly reduces synchronization complexity, minimizes frontend overhead, and allows the browser

runtime to remain lightweight and deterministic

5.1 Static Projection Runtime Generation

During startup, the display server composes the complete browser projection runtime from registered runtime components.

These components may include:

- HTML template definitions
- CSS styling rules
- event anchor registrations
- reactive stream bindings
- projection runtime scripts
- synchronization infrastructure
- and optional runtime extensions

The resulting projection runtime is emitted as a complete static browser environment served directly by the display server. The startup sequence may therefore be represented conceptually as:

Display Server Startup ->
Component Registration ->
HTML Composition ->
CSS Composition ->
Projection Runtime Composition ->
Browser Runtime Generation ->
Browser Connection ->
Runtime Synchronization

Within this architecture, browser clients receive a stable projection runtime environment before synchronization begins. Templates, styles, event bindings, and projection runtime logic are therefore not synchronized dynamically through runtime update streams. Instead, they form the static execution environment of the projection runtime itself.

Changes to projection runtime structure typically require only a browser refresh after the display server has regenerated the runtime environment. This model intentionally prioritizes runtime simplicity and deterministic synchronization behavior over dynamic frontend reconfiguration

5.2 Projection Runtime Responsibilities

The browser projection runtime fulfills a narrowly defined set of responsibilities.

These responsibilities include:

- establishing synchronization channels
- maintaining handle-based element lookup tables
- instantiating synchronized elements from registered templates
- processing structural synchronization operations
- forwarding interaction events
- maintaining lightweight local projection state
- and exposing reactive event streams

Importantly, the projection runtime does not own:

- authoritative application state
- application business logic
- runtime orchestration
- or long-term session ownership

This distinction preserves deterministic runtime behavior and prevents synchronization divergence between runtime systems and browser projections. The browser runtime therefore behaves more similarly to a lightweight synchronized compositor than to a traditional browser application framework

5.3 Template Registry Architecture

Projection runtimes instantiate synchronized structures through locally registered templates. Modern browser environments provide HTML5 template elements capable of storing pre-parsed DOM structures separately from active rendering trees. Template contents may later be cloned efficiently without repeatedly constructing complex structures through imperative DOM APIs.

Compared to repeated runtime element construction through functions such as `createElement`, template cloning significantly reduces:

- repeated parsing overhead
- allocation pressure
- layout invalidation frequency
- and DOM construction cost

The display server therefore generates a template registry during runtime composition. Each registered template receives a stable numeric identifier used by the synchronization layer during append operations.

Projection runtimes resolve incoming element identifiers against this template registry before cloning synchronized structures into the retained projection tree. Because templates are statically registered during runtime generation, synchronization payloads remain compact and independent from raw HTML serialization

5.4 Structural Patch Processing

Runtime synchronization occurs through incremental structural patch operations operating on retained projection graphs. Projection runtimes maintain local lookup structures mapping runtime handles to active DOM nodes via HTML id attributes.

Incoming synchronization batches are processed sequentially through a lightweight patch dispatcher.

Example:

```
for (const [command, args] of batch)
{
  switch (command)
  {
    case "append":
      append(args);
      break;

    case "move":
      move(args);
      break;

    case "modify":
      modify(args);
      break;
```

```

    case "remove":
        remove(args);
        break;
    }
}

```

Because all synchronized structures share the same generalized handle model, normal elements and composable surfaces may be processed through identical synchronization logic.

The synchronization layer therefore remains:

- renderer-independent
- lightweight
- deterministic
- and structurally minimal

5.5 Reactive Event Streams

Rather than exposing traditional fragmented event listener architectures, the projection runtime translates browser interaction events into reactive event streams.

Each event category exposed by the projection runtime provides a dedicated stream capable of transporting:

- the originating element handle
- event identifiers
- synchronized parameters
- projection metadata
- and runtime-specific interaction information

Examples may include:

- pointer streams
- keyboard streams
- focus streams
- surface lifecycle streams
- resize streams
- and gesture streams

A synchronized event payload may conceptually resemble:

```

{
  "handle": 4096,
  "event": 5,
  "parameters": {
    "x": 120,
    "y": 320
  }
}

```

This model allows interaction handling, synchronization behavior, runtime composition, and higher-level orchestration systems to be expressed through composable reactive pipelines. Because the surrounding runtime ecosystem already exposes Reactive Extensions as modular runtime packages, projection streams integrate naturally into existing runtime composition systems

5.6 Local Projection State and Roundtrip Avoidance

Certain categories of interaction events require immediate local visual response in order to preserve responsiveness.

Operations such as:

- text input
- cursor movement
- selection updates
- focus transitions
- and pointer interaction

may become visually degraded if every modification depends entirely on remote synchronization roundtrips. The projection runtime therefore maintains a lightweight temporary local projection state for latency-sensitive interactions.

Input changes may be applied locally while simultaneously being forwarded to the runtime synchronization layer. This behavior resembles controlled optimistic projection updates while preserving authoritative runtime ownership.

Locally applied modifications may optionally be marked explicitly within synchronization metadata in order to prevent redundant state echoing during subsequent runtime synchronization. This approach preserves responsive local interaction behavior while maintaining deterministic runtime synchronization semantics

5.7 Browser Runtime Composition

The browser projection runtime itself is distributed as part of the display server package.

A minimal default runtime implementation may already provide:

- template registration
- synchronization handling
- event stream infrastructure
- projection graph management
- and browser integration behavior

Applications may either use the default runtime directly or modify individual runtime components before compilation. Because the projection runtime is generated during display server startup, runtime composition remains highly modular while avoiding the operational complexity associated with heavyweight frontend frameworks or embedded browser runtimes.

The resulting browser runtime may be:

- embedded directly into the display server
- compiled into static runtime resources
- or generated dynamically during startup

This architecture preserves lightweight deployment behavior while maintaining full runtime configurability

5.8 Runtime Layer Separation

The browser projection architecture is fundamentally divided into three distinct runtime layers

Static Projection Layer

The static projection layer contains:

- templates
- CSS rules
- event bindings
- reactive stream definitions
- projection runtime infrastructure
- and browser integration logic

This layer is generated during display server startup and remains stable throughout runtime execution

Dynamic Synchronization Layer

The synchronization layer transports:

- structural patch operations
- runtime state updates
- projection changes
- and synchronization metadata

This layer operates continuously throughout runtime execution

Reactive Event Layer

The reactive event layer translates browser interaction events into synchronized runtime streams. This layer bridges browser

interaction behavior with runtime-owned orchestration systems while preserving deterministic synchronization semantics.

The strict separation between these layers allows the browser runtime to remain lightweight, deterministic, modular, and independent from application-specific frontend frameworks.

The following chapters examine browser engine integration behavior and the platform-specific implementation details required to support synchronized projection runtimes across Chromium- and Gecko-based browser environments.

6. Browser Integration and Runtime Hosting

The browser projection architecture intentionally avoids introducing custom embedded rendering runtimes or heavyweight browser abstraction frameworks. Instead, the system directly utilizes already installed system browsers as standardized rendering environments. This approach significantly reduces deployment complexity while simultaneously leveraging mature browser engines that already provide:

- hardware accelerated rendering
- input handling
- accessibility support
- touch interaction
- network infrastructure
- security isolation
- mobile compatibility

- fullscreen support
- and modern web platform APIs

Rather than replacing browser runtimes, the display server instead treats existing browser engines as externally hosted synchronized projection environments. This distinction is fundamental to the architecture.

The system does not attempt to behave as:

- a custom browser engine
- an embedded web renderer
- or an Electron-style application runtime

Instead, the display server coordinates synchronized projection sessions across independently hosted browser runtimes

6.1 Browser-Based Projection Environments

Projection runtimes may operate within:

- desktop browser windows
- application-style browser windows
- mobile browsers
- fullscreen sessions
- kiosk-style environments
- remote projection displays
- or headless browser environments

Because the synchronization architecture is browser-engine independent, the same display server may expose synchronized

projection environments simultaneously across:

- Chromium-based browsers
- Gecko-based browsers
- desktop systems
- tablets
- and mobile devices

Each browser instance effectively behaves as a synchronized projection node attached to the display server. This architecture allows runtime surfaces to remain accessible from multiple independent devices without introducing platform-specific frontend implementations

6.2 Launching Browser App Windows

Modern browser engines provide several mechanisms allowing browser sessions to behave similarly to standalone application windows. These mechanisms generally disable or minimize traditional browser chrome including:

- navigation bars
- tab controls
- bookmark interfaces
- extension toolbars
- and browser menus

The display server may optionally launch projection sessions automatically using browser-specific startup parameters

Chromium-based browsers provide native application window support through the `--app` parameter.

Example:

```
chrome --app=https://localhost:8080
```

This launches the synchronized projection environment inside a dedicated application-style window without standard browser navigation elements. Additional parameters may further reduce browser UI visibility:

```
chrome \  
--app=https://localhost:8080 \  
--disable-session-crashed-bubble \  
--disable-infobars \  
--disable-features=TranslateUI
```

Firefox and other Gecko-based browsers expose fewer dedicated application-window features compared to Chromium-based engines. Rather than relying on fullscreen kiosk hosting, Gecko runtime integration is more reliably implemented through isolated runtime profiles combined with browser chrome reduction. Projection runtimes may therefore generate temporary runtime-specific Firefox profiles during display server startup.

A generated runtime profile may contain:

- runtime-specific browser preferences
- projection-oriented browser UI overrides

- Wake Lock configuration
- fullscreen permissions
- synchronization policies
- and runtime-specific extension configuration

Firefox runtime sessions may then be launched through:

```
firefox \  
--no-remote \  
--profile ./runtime-profile \  
--new-window https://localhost:8080
```

The `--no-remote` parameter is particularly important because Firefox otherwise tends to attach new windows to already running browser sessions. This prevents runtime isolation and complicates multi-session projection environments. Firefox browser chrome reduction may additionally be implemented through a generated:

```
chrome/userChrome.css
```

inside the runtime profile.

Example:

```
#TabsToolbar  
{  
    visibility: collapse !important;  
}  
  
#nav-bar
```

```
{  
  visibility: collapse !important;  
}
```

```
#PersonalToolbar  
{  
  visibility: collapse !important;  
}
```

```
#sidebar-box  
{  
  visibility: collapse !important;  
}
```

Custom browser stylesheet support must additionally be enabled through:

```
user_pref(  
  
  "toolkit.legacyUserProfileCustomizations.stylesheets",  
  true  
);
```

inside the generated:

```
prefs.js
```

runtime profile configuration.

This approach allows Gecko-based runtimes to behave similarly to lightweight synchronized application windows while remaining fully standards-compliant browser sessions. Unlike Chromium application windows, Firefox still internally behaves as a complete browser

environment. However, the visible runtime surface may be reduced substantially.

Popup-based runtime hosting may additionally be implemented through browser-side runtime initialization logic.

Example:

```
window.open(  
  "https://localhost:8080",  
  "projection",  
  "popup,width=1280,height=720"  
);
```

Because browser capabilities differ significantly between engines, the display server architecture intentionally treats browser windows as loosely standardized projection environments rather than fully identical runtime hosts

6.3 Chromium Runtime Integration

Chromium-based browsers currently provide the most complete runtime hosting feature set for browser-based synchronized projection systems.

Available functionality includes:

- dedicated application windows
- fullscreen projection support
- stable wake lock behavior
- progressive web application integration
- advanced compositor acceleration
- robust WebSocket support

- and mature mobile compatibility

Chromium browsers additionally expose predictable window-management behavior across desktop operating systems. This allows synchronized projection environments to behave similarly to lightweight native desktop applications while still remaining fully browser-hosted.

The display server architecture does not rely on Chromium-specific rendering behavior. However, Chromium engines currently provide the most complete implementation environment for browser-hosted synchronized runtimes

6.4 Gecko Runtime Integration

Gecko-based browsers provide several architectural advantages including strong privacy tooling, extension flexibility, and broad platform availability. However, Gecko runtime hosting behavior differs significantly from Chromium-based browsers.

Firefox currently exposes fewer dedicated application-window oriented runtime hosting features and places greater emphasis on traditional browser interaction models.

This affects:

- standalone application window behavior
- popup persistence
- fullscreen integration
- browser chrome visibility
- extension integration
- and runtime-oriented window management

Despite these differences, Gecko-based browsers remain fully capable synchronized projection environments. Projection runtimes hosted inside Firefox continue to support:

- WebSocket synchronization
- fullscreen projection
- reactive event streams
- touch interaction
- mobile projection workflows
- Wake Lock support
- and hardware accelerated rendering

The architecture therefore intentionally avoids dependence on browser-engine-specific runtime assumptions. Instead, the synchronization layer only requires:

- standards-compliant rendering behavior
- modern JavaScript execution
- DOM template support
- WebSocket communication
- and modern browser API availability

This preserves portability across both Chromium- and Gecko-based environments. Firefox projection hosting is therefore implemented primarily through runtime-generated isolated browser profiles combined with projection-oriented browser chrome reduction.

This strategy avoids dependence on:

- experimental browser features
- undocumented startup modes
- or fullscreen-only kiosk environments

Projection runtimes may therefore remain compatible with conventional desktop multitasking workflows while still behaving similarly to lightweight synchronized application windows

6.5 Mobile Browser Integration

Because the projection architecture relies entirely on standard browser runtimes, synchronized surfaces may be accessed directly from mobile devices without requiring separate native application implementations.

This is particularly important for:

- remote tooling environments
- secondary control surfaces
- tablet-based dashboards
- streamer control clients
- portable runtime management panels
- and auxiliary interaction devices

The display server may therefore expose synchronized projection environments directly over local networks through HTTPS-secured browser sessions. Mobile browsers effectively behave as additional synchronized display nodes connected to the retained runtime graph.

This architecture enables lightweight multi-device projection workflows without introducing separate mobile frontend stacks

6.6 Wake Lock and Long-Lived Sessions

Long-lived synchronized projection environments may require prevention of automatic display suspension or device sleep behavior. Modern browser environments expose the Screen Wake Lock API allowing browser runtimes to request temporary prevention of automatic display sleep.

Example:

```
let wakeLock = null;

async function acquireWakeLock()
{
  try
  {
    wakeLock = await
navigator.wakeLock.request("screen");
  }
  catch (error)
  {
    console.error(error);
  }
}
```

Wake Lock functionality is particularly important for:

- dashboard displays
- remote control panels
- mobile projection devices
- fullscreen control surfaces

- and long-running synchronized sessions

Projection runtimes may negotiate Wake Lock availability during capability negotiation. Because browser support differs between engines and platforms, Wake Lock integration remains capability-driven rather than mandatory

6.7 Local Network Projection Access

Display servers may expose synchronized projection environments directly through local network HTTPS endpoints.

This allows browser runtimes hosted on:

- desktop systems
- tablets
- laptops
- or mobile devices

to connect to synchronized runtime surfaces without requiring cloud infrastructure or externally hosted frontend deployments.

Authentication may be performed through:

- session tokens
- application passwords
- runtime-generated access keys
- or local network trust policies

Because the browser runtime itself remains lightweight and stateless, remote devices effectively behave as temporary synchronized projection clients.

This architecture is especially useful for:

- local development environments
- remote tooling systems
- portable dashboards
- streamer control interfaces
- and multi-device runtime management

6.8 Security and Runtime Isolation

The architecture intentionally leverages existing browser security infrastructure instead of bypassing it through embedded rendering runtimes.

Browser engines already provide:

- process isolation
- sandboxing
- permission systems
- origin restrictions
- HTTPS enforcement
- and resource isolation

The display server therefore benefits directly from mature browser security models while avoiding many security concerns commonly associated with custom embedded web runtimes. Because browser runtimes remain externally hosted, the display server additionally preserves strong separation between:

- runtime orchestration
- projection rendering

- and client interaction environments

This separation aligns naturally with the modular runtime architecture used throughout the surrounding runtime ecosystem

6.9 Headless and Embedded Runtime Modes

Although the primary architecture targets externally hosted browser runtimes, the synchronization system may also operate within headless browser environments.

Headless execution is particularly useful for:

- automated testing
- runtime validation
- surface snapshot generation
- projection verification
- and server-side rendering workflows

Chromium-based browsers provide mature headless runtime support.

Example:

```
chrome \
--headless=new \
--disable-gpu \
--remote-debugging-port=9222 \
https://localhost:8080
```

Firefox additionally exposes headless execution support.

Example:

```
firefox --headless https://localhost:8080
```

However, browser-specific runtime behavior may still differ between engines, particularly regarding fullscreen support, extension behavior, compositor acceleration, and runtime lifecycle management. The display server therefore treats headless execution primarily as an auxiliary runtime environment rather than as the primary deployment target

7. Roadmap and Extensibility

The projection architecture described throughout this work intentionally focuses on establishing a lightweight, modular, and browser-independent synchronized runtime foundation. Rather than attempting to provide a complete monolithic frontend framework, the system instead defines a generalized projection model capable of supporting a wide range of higher-level runtime systems.

Because the architecture separates:

- runtime ownership
- synchronization
- projection composition
- browser integration
- and reactive interaction flows

additional runtime systems may be layered on top of the synchronization model without fundamentally changing the underlying runtime architecture.

Potential future runtime extensions include:

- popup and overlay systems
- docking environments
- graph-oriented runtime projections
- visual scripting systems
- workspace-oriented tooling surfaces
- multi-device runtime dashboards
- remote control interfaces
- collaborative projection environments
- and web-based desktop-style runtime systems

The generalized handle model, retained projection graph, and reactive synchronization layer provide a stable foundation upon which these higher-level runtime environments may be implemented incrementally. Because projection runtimes remain browser-hosted and synchronization-oriented, advanced runtime systems may additionally operate across:

- desktop systems
- mobile devices
- tablets
- remote development environments
- and distributed runtime sessions

without requiring separate frontend implementations. This extensibility is a direct consequence of the intentionally modular runtime architecture used throughout the surrounding ecosystem.

8. Summary and Vision

Modern application development continues to suffer from fragmentation between native UI frameworks, browser runtimes, platform-specific rendering systems, and increasingly heavyweight frontend ecosystems. The architecture presented throughout this work approaches the problem from a different perspective.

Instead of treating the browser as the application itself, the browser becomes a universally available synchronized projection runtime attached to a runtime-owned display server. This distinction fundamentally changes the role of the browser within the system.

The browser runtime no longer owns application state, orchestration logic, or runtime authority. Instead, it behaves as a lightweight synchronized projection environment capable of rendering retained runtime surfaces and forwarding interaction streams.

By leveraging:

- standardized browser technologies
- lightweight synchronization protocols
- retained projection graphs
- reactive runtime streams
- and modular runtime composition

the architecture enables browser-hosted runtime environments without introducing embedded browser runtimes or heavyweight frontend abstraction layers.

This allows synchronized runtime systems to remain:

- lightweight
- modular
- browser-independent
- remotely accessible
- multi-device capable
- and incrementally extensible

The long-term vision of the architecture is not the creation of yet another browser framework. Instead, the goal is the establishment of a generalized synchronized runtime projection system capable of supporting tooling environments, runtime dashboards, editor systems, visual scripting environments, remote interaction surfaces, and future browser-hosted runtime ecosystems through a unified synchronization architecture.

As browser capabilities continue to evolve across desktop and mobile environments, the browser increasingly becomes not merely a document renderer, but a universally available runtime projection environment. The architecture presented throughout this work attempts to utilize that capability directly while preserving runtime ownership, modularity, deterministic synchronization, and deployment simplicity